Plan work with GitHub Issues and MCP

Session M7-03 | Product and Delivery Teams | Intermediate

Turn the brief into GitHub issues

Session M7-02 produced an approved decision brief.

Session M7-03 turns it into real GitHub issues.

This is the third step in **Delivery Decision Studio**.

MCP in plain words

MCP (Model Context Protocol) is a standard way to plug tools into Copilot.
An **MCP server** exposes named tools, such as "get issue" or "create issue".
Copilot calls them for you.

Think of a USB port: one plug shape, many devices.

You use the tools in this session. You don't build a server.

Tool discovery and permissions

Discovery: ask Copilot to list the GitHub tools in this session. Only listed tools can be called.

Permissions: a tool acts with the rights of the connection. If your account can't write issues, the write fails. If it can read a private repository, Copilot can read it too.

- Read tools: search issues, list labels and milestones.
- Write tools: create or update an issue.

Never paste a token into a prompt or file.

GitHub MCP or the app's own features?

	GitHub MCP	App-native (UI, `gh`, built-in context)
Best for	Structured reads and writes Copilot can plan and repeat	One-off clicks, quick looks
Setup	Needs an active connection and permissions	Already there
Review	Tool call shows exact arguments	You see the result, not the call
Risk	A write can run at scale	Slower, manual

Use MCP when Copilot must read or write GitHub as part of a plan. Use the web UI when you just need to look at one issue.

Write issues others can act on

The issue should let a contributor decide whether they can start, what success looks like, and who will review it.

Issue element	What it prevents
Outcome	Work that repeats a vague request
Results needed for acceptance	"Done" without a way to prove it
Non-goal	Scope growth during implementation
Dependency	Work that starts before a needed decision
Ownership gap	Silent assumptions about who decides

Write enough detail for the next contributor to act.

Parent and child issues answer different questions

A **parent issue** holds the outcome. **Child issues** (sub-issues) are the pieces of work under it. **Labels** classify issues. A **milestone** groups issues under a target. Use only labels and milestones that already exist.

The parent issue holds the outcome, shared constraints, and review path. Each child issue owns one independently reviewable result.

```
Parent: Improve request-status visibility
  ├─ Child: Public status mapping
  ├─ Child: Requester status view
  ├─ Child: Empty and stale states
  └─ Child: Requester-access verification
```

If two child issues must edit the same behavior at the same time, sequence them or clarify the design first.

Review each MCP write

Approve a write only after you see the target repository and the full result.

After the write, fetch the issue again. Compare what GitHub stored with the plan, not with the tool's success message.

The Markdown plan is a temporary preview. **The created GitHub issues are authoritative.**

Read before writing

1. Check the repository.
2. Inspect existing labels and milestones.
3. Search for duplicate issues.
4. Read the approved brief.
5. Report permission or context uncertainty.

Decide what to do with existing work

The lab may ship `existing-issues.json`. It is only a fallback for repositories without seeded issues. The lesson is reading live GitHub state.

Candidate	Decision
Exact duplicate	Reuse or close according to repository policy
Partial overlap	Extend the existing issue and create only the missing work
Conflict	Keep separate and record why
Unrelated	Leave unchanged

The lab includes open, closed, overlapping, and conflicting status issues.

Decompose by outcome

Create one parent issue and child issues with defined scope.

Each child needs:

- an independently reviewable result;
- results needed for acceptance;
- a non-goal;
- dependencies;
- an owner or visible ownership gap.

Preview the complete write

Before approval, show:

- repository owner and name;
- issue count;
- every title and body;
- labels and milestone;
- relationships;
- fields left unset.

When MCP fails

Symptom	Likely cause	Fix
No GitHub tools listed	Server not installed or not running	**Customize → MCP → Installed**
Read works, write fails	Account lacks issue-write access	Ask for access or stop
Wrong repository	Prompt didn't name the repo	Reject and restate owner and name
"Success" but no issue	Silent or partial failure	Fresh read of the repository
Label or milestone error	Name doesn't exist	Use an existing one or leave it unset

Check what GitHub saved

After the write:

1. start a fresh request;
2. retrieve the issues;
3. compare them with the approved plan;
4. correct one defect;
5. retrieve the corrected issue again.

Lab result

- Temporary reviewed issue preview
- Authoritative live issue set
- One parent issue
- Four required child roles; an optional fifth stays outside Session M7-05
- Saved issues checked through a fresh GitHub read
- Parent-to-child links that Session M7-04 reads directly

Stop if you cannot review the tool call or check the saved issues.